

while loops

while loops are a while to have your program continuously run some block of code until a condition is met (made TRUE). The syntax is:

```
while (condition){  
  # Code executed here  
  # while condition is true  
}
```

A major concern when working with a while loop is to make sure that at some point the condition should become true, otherwise the while loop will go forever! Remember you can use Ctrl-C to kill a process in R Studio!

Here's a real quick side note on printing variables along with strings:

In [6]:

```
print('Just a string')
```

```
[1] "Just a string"
```

In [7]:

```
var <- 'a variable'  
cat('My variable is: ',var)
```

```
My variable is:  a variable
```

In [9]:

```
var <- 25  
cat('My number is:',var)
```

```
My number is: 25
```

In [12]:

```
# Could also use:  
print(paste0("Variable is: ", var))
```

```
[1] "Variable is: 25"
```

We'll be using the **cat** function in the next example which shows how a while loop works:

In [14]:

```
x <- 0  
  
while(x < 10){  
  
  cat('x is currently: ',x)  
  print(' x is still less than 10, adding 1 to x')  
  
  # add one to x  
  x <- x+1  
}
```

```
x is currently: 0 [1] " x is still less than 10, adding 1 to x"
```

```
x is currently: 0[1] " x is still less than 10, adding 1 to x"
x is currently: 1[1] " x is still less than 10, adding 1 to x"
x is currently: 2[1] " x is still less than 10, adding 1 to x"
x is currently: 3[1] " x is still less than 10, adding 1 to x"
x is currently: 4[1] " x is still less than 10, adding 1 to x"
x is currently: 5[1] " x is still less than 10, adding 1 to x"
x is currently: 6[1] " x is still less than 10, adding 1 to x"
x is currently: 7[1] " x is still less than 10, adding 1 to x"
x is currently: 8[1] " x is still less than 10, adding 1 to x"
x is currently: 9[1] " x is still less than 10, adding 1 to x"
```

Let's add an if statement to this logic!

In [15]:

```
x <- 0

while(x < 10){

  cat('x is currently: ',x)
  print(' x is still less than 10, adding 1 to x')

  # add one to x
  x <- x+1
  if(x==10){
    print("x is equal to 10! Terminating loop")
  }
}
```

```
x is currently: 0[1] " x is still less than 10, adding 1 to x"
x is currently: 1[1] " x is still less than 10, adding 1 to x"
x is currently: 2[1] " x is still less than 10, adding 1 to x"
x is currently: 3[1] " x is still less than 10, adding 1 to x"
x is currently: 4[1] " x is still less than 10, adding 1 to x"
x is currently: 5[1] " x is still less than 10, adding 1 to x"
x is currently: 6[1] " x is still less than 10, adding 1 to x"
x is currently: 7[1] " x is still less than 10, adding 1 to x"
x is currently: 8[1] " x is still less than 10, adding 1 to x"
x is currently: 9[1] " x is still less than 10, adding 1 to x"
[1] "x is equal to 10! Terminating loop"
```

break

You can use **break** to break out of a loop. Previously we showed an if statement checking for 10, but this wasn't actually stopping the loop, the while condition check on the next run stopped the loop, let's show an example where we could use **break** to terminate the loop:

In [17]:

```
x <- 0

while(x < 10){

  cat('x is currently: ',x)
  print(' x is still less than 10, adding 1 to x')

  # add one to x
  x <- x+1
  if(x==10){
    print("x is equal to 10!")
    print("I will also print, woohoo!")
  }
}
```

```
x is currently: 0[1] " x is still less than 10, adding 1 to x"
x is currently: 1[1] " x is still less than 10, adding 1 to x"
x is currently: 2[1] " x is still less than 10, adding 1 to x"
x is currently: 3[1] " x is still less than 10, adding 1 to x"
x is currently: 4[1] " x is still less than 10, adding 1 to x"
```

```
x is currently: 5[1] " x is still less than 10, adding 1 to x"
x is currently: 6[1] " x is still less than 10, adding 1 to x"
x is currently: 7[1] " x is still less than 10, adding 1 to x"
x is currently: 8[1] " x is still less than 10, adding 1 to x"
x is currently: 9[1] " x is still less than 10, adding 1 to x"
[1] "x is equal to 10!"
[1] "I will also print, woohoo!"
```

In [18]:

```
x <- 0

while(x < 10){

  cat('x is currently: ',x)
  print(' x is still less than 10, adding 1 to x')

  # add one to x
  x <- x+1
  if(x==10){
    print("x is equal to 10!")
    break
    print("I will also print, woohoo!")
  }
}
```

```
x is currently: 0[1] " x is still less than 10, adding 1 to x"
x is currently: 1[1] " x is still less than 10, adding 1 to x"
x is currently: 2[1] " x is still less than 10, adding 1 to x"
x is currently: 3[1] " x is still less than 10, adding 1 to x"
x is currently: 4[1] " x is still less than 10, adding 1 to x"
x is currently: 5[1] " x is still less than 10, adding 1 to x"
x is currently: 6[1] " x is still less than 10, adding 1 to x"
x is currently: 7[1] " x is still less than 10, adding 1 to x"
x is currently: 8[1] " x is still less than 10, adding 1 to x"
x is currently: 9[1] " x is still less than 10, adding 1 to x"
[1] "x is equal to 10!"
```

Notice how the **break** stopped the "I will also print, woohoo!" from occurring. You can use this as an internal check inside a while loop to stop is based off some other condition.

Up next let's learn about the **for loop**, after that, we will test your knowledge!